

Synchronisation GitHub → GitHub (deux dépôts)

Procédure pour que chaque `git push` envoie automatiquement le code vers **deux dépôts GitHub distincts** en même temps (ex. un dépôt personnel et un dépôt d'organisation, ou deux comptes différents).

1. Prérequis

- Le premier dépôt existe déjà sur GitHub 
- Le second dépôt existe déjà sur GitHub  (ou demander à l'admin de le créer / donner accès)

2. Configurer l'authentification SSH pour les deux dépôts

Si les deux dépôts sont sur des comptes GitHub différents, il faut généralement deux clés SSH distinctes (une clé ne peut être associée qu'à un seul compte GitHub à la fois pour une authentification simple).

```
bash

# Vérifier les clés SSH existantes
ls -al ~/.ssh

# Générer une clé pour le premier compte/dépôt
ssh-keygen -t ed25519 -C "compte1@example.com" -f ~/.ssh/id_ed25519_compte1

# Générer une clé pour le second compte/dépôt
ssh-keygen -t ed25519 -C "compte2@example.com" -f ~/.ssh/id_ed25519_compte2

# Afficher les clés publiques à copier
cat ~/.ssh/id_ed25519_compte1.pub
cat ~/.ssh/id_ed25519_compte2.pub
```

Ajouter chaque clé publique dans le compte GitHub correspondant : **avatar** → **Settings** → **SSH and GPG keys** → **New SSH key**

Configurer un alias SSH par compte

Éditer (ou créer) `~/.ssh/config`:

```
Host github-compte1
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_compte1

Host github-compte2
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_compte2
```

Tester les deux connexions :

```
bash

ssh -T git@github-compte1
ssh -T git@github-compte2
```

Un message de bienvenue doit confirmer l'authentification pour chacun des deux comptes.

Si les deux dépôts appartiennent au **même compte** GitHub, une seule clé SSH suffit et cette étape d'alias n'est pas nécessaire — passer directement à l'étape 3 avec `git@github.com:<user>/<repo>.git` pour les deux URLs.

3. Configurer le double push en local

Dans le dossier du projet déjà cloné depuis le premier dépôt :

```
bash

# Vérifier l'état actuel
git remote -v

# Ajouter les deux URLs de push sur origin
git remote set-url --add --push origin git@github-compte1:<user1>/<repo1>.git
git remote set-url --add --push origin git@github-compte2:<user2>/<repo2>.git

# Vérifier la config finale
git remote -v
```

Résultat attendu :

```
origin  git@github-compte1:<user1>/<repo1>.git (fetch)
origin  git@github-compte1:<user1>/<repo1>.git (push)
origin  git@github-compte2:<user2>/<repo2>.git (push)
```

4. Vérifier les droits d'accès sur les deux dépôts

L'authentification SSH ne suffit pas : il faut aussi avoir le **droit d'écriture (Write)** sur chacun des deux dépôts, surtout s'il s'agit de dépôts d'organisation.

Si besoin, demander à l'admin du dépôt concerné d'ajouter le compte dans **Settings → Collaborators and teams**, avec le rôle **Write** minimum.

5. Usage quotidien

```
bash

git add .
git commit -m "mon message"
git push origin <nom-de-ta-branche>
```

Ou, si la branche est déjà suivie (upstream défini) :

```
bash
```

```
git push
```

Chaque push envoie automatiquement le code vers **les deux dépôts GitHub** en même temps.

⚠ Points de vigilance

- Si le push vers l'un des deux dépôts échoue, l'autre peut quand même réussir → toujours vérifier la sortie complète du terminal.
- Si les historiques des deux dépôts divergent (commits différents), le push peut être rejeté (`(non-fast-forward)`) → il faudra les synchroniser manuellement.
- Cette configuration est **locale à la machine** : si un collègue push directement, il doit avoir la même configuration (remotes + clés SSH) pour synchroniser aussi de son côté.
- Attention aux clés SSH : si les deux dépôts sont sur des comptes différents, bien vérifier que chaque alias (`(github-compte1)`, `(github-compte2)`) pointe vers la bonne clé, sinon GitHub refusera l'accès en écriture ou poussera avec la mauvaise identité.